

فاز ۱۶: تست‌نویسی (Testing) – کامل و عمیق

تست‌نویسی یکی از مهم‌ترین مهارت‌های یه توسعه‌دهنده حرفه‌ای هست. توی مصاحبه‌های شغلی، ازت انتظار می‌ره که حداقل با مفاهیم پایه تست و ابزارهای آشنا باشی.

۱۶.۱ چرا تست‌نویسی؟

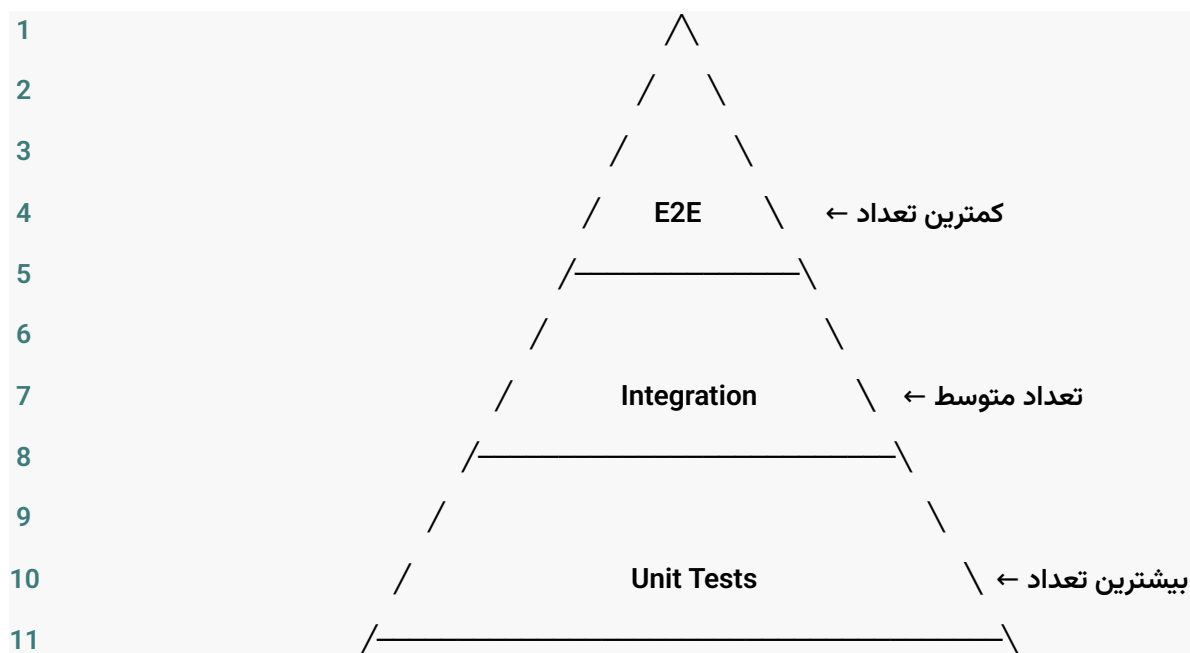
سوال مصاحبه‌ای: چرا باید تست بنویسیم؟

جواب:

۱. اطمینان از صحت کد: مطمئن میشیم کدمون درست کار می‌کنه
۲. جلوگیری از Regression Bug: تغییرات جدید، کد قدیمی رو نمی‌شکنه
۳. مستندسازی: تست‌ها به عنوان مستندات زنده عمل می‌کنن
۴. طراحی بهتر: کد تست‌پذیر، معمولاً طراحی بهتری داره
۵. اعتماد به نفس: با خیال راحت می‌تونی کد رو تغییر بدی

۱۶.۲ انواع تست از نظر سطح (Test Pyramid)

هرم تست (Test Pyramid) به ما می‌گه چه مقدار از هر نوع تست باید داشته باشیم:



جدول انواع تست

نوع تست	توضیح	مثال	تعداد
Unit Test	تست یک واحد کوچک (متد یا کلاس) به تنهایی	تست کردن <code>calculateSalary()</code>	زیاد ✓
Integration Test	تست ارتباط چند جزء با هم	تست <code>UserService</code> با <code>UserRepository</code> و واقعی	متوسط
E2E Test	تست کل سیستم از اول تا آخر	ثبت نام ← لاگین ← خرید ← خروج	کم

۱۶.۳ اصول طلایی تست (اصول FIRST)

این اصول پایه‌ای‌ترین چیزهایی هستند که باید در مورد تست بدونی:

مثال	توضیح کامل	اصل
تست‌های Unit نباید به دیتابیس یا شبکه وصل بشن	تست‌ها باید خیلی سریع اجرا بشن تا بتونی مرتب اجراشون کنی	Fast (سریع)
هر تست محیط خودش رو آماده کنه و بعد پاک کنه	هر تست نباید به ترتیب یا نتیجه تست دیگه وابسته باشه	Independent (مستقل)
از مقادیر ثابت استفاده کن، نه تاریخ و زمان جاری	هر بار که تست اجرا بشه، نتیجه یکسان باشه	Repeatable (تکرارپذیر)
استفاده از assert، نه اینکه خروجی رو چشمی چک کنی	تست باید خودش تشخیص بده قبول شده یا رد شده	Self-Validating (خودتأییدگر)
TDD: اول تست، بعد کد	تست رو قبل یا هم‌زمان با کد بنویس	Timely (به‌موقع)

۱۶.۴ ساختار Given-When-Then (AAA)

این ساختار استاندارد نوشتن تست‌هاست و باعث میشه تست‌ها خواناتر بشن:

```

1 @Test
2 void shouldReturnUserWhenValidUsernameProvided() {
3     // GIVEN (یا Arrange) - آماده‌سازی
4     UserService service = new UserService();
5     String username = "ali";
6
7     // WHEN (یا Act) - اجرای عملیات
8     User result = service.findUser(username);
9
10    // THEN (یا Assert) - تأیید نتیجه
11    assertNotNull(result);
12    assertEquals("ali", result.getUsername());
13 }
```

نام‌های دیگه این ساختار:

Arrange-Act-Assert (AAA)•

Given-When-Then (GWT)•

۱۶.۵ تکنیک‌های طراحی تست

۱. Boundary Value Analysis (تحلیل مرزها)

قانون: خطاها معمولاً در مرزهای ورودی رخ میدن. همیشه مرزها رو تست کن.

```

1 // فقط ۱۸ تا ۶۰ رو قبول می‌کنه acceptAge(int age) متد a.
2
3 @Test
4 void testAgeBoundaries() {
5     // مرزهای پایین
6     assertFalse validator.acceptAge(17); // دقیقاً زیر مرز
7     assertTrue validator.acceptAge(18); // درست روی مرز پایین
8
9     // مرزهای بالا
10    assertTrue validator.acceptAge(60); // درست روی مرز بالا
11    assertFalse validator.acceptAge(61); // دقیقاً بالای مرز
12
13    // داخل بازه
14    assertTrue validator.acceptAge(30);
15    assertTrue validator.acceptAge(45);
16 }
```

۲. Equivalence Partitioning (تقسیم به کلاس‌های هم‌ارز)

قانون: ورودی‌ها رو به دسته‌هایی تقسیم کن که رفتار مشابهی دارن. از هر دسته فقط یکی رو تست کن.

```

1 // b. براساس سن تخفیف میدہ calculateDiscount(int age) متد
2
3 @Test
4 void testDiscountByAgeGroups() {
5     // دسته کودک (کمتر از ۱۲) - یک نمونه کافیه
6     assertEquals(0.5, calculator.calculateDiscount(5));
7
8     // دسته بزرگسال (۱۲ تا ۶۰)
9     assertEquals(0.0, calculator.calculateDiscount(25));
10
11    // دسته سالمند (بیشتر از ۶۰)
12    assertEquals(0.3, calculator.calculateDiscount(70));
13 }
```

۳. Negative Testing (تست منفی)

قانون: فقط مسیر خوش‌بخت رو تست نکن. ورودی‌های غلط رو هم تست کن.

```

1 @Test
2 void testNegativeScenarios() {
3     UserService service = new UserService();
4
5     // ورودی null
6     assertThrows(IllegalArgumentException.class,
```

```

7    () -> service.registerUser(null));
8
9    // ورودی خالی
10   User emptyUser = new User("", "");
11   assertThrows(ValidationException.class,
12       () -> service.registerUser(emptyUser));
13
14   // پسورد خیلی کوتاه
15   User weakUser = new User("ali", "123");
16   assertFalse(service.registerUser(weakUser));
17
18   // کاربر تکراری
19   service.registerUser(new User("ali", "pass123"));
20   assertThrows(DuplicateUserException.class,
21       () -> service.registerUser(new User("ali", "pass456")));
22 }

```

۴. اصل تک‌مسئولیتی در تست

قانون: هر تست فقط یک رفتار رو بررسی کنه.

```

1 // a. ❌ چند تا - چند assert با موضوعات مختلف
2 @Test
3 void testUserService() {
4     User user = service.register("ali", "pass");
5     assertNotNull(user);
6     assertEquals("ali", user.getUsername());

```

```
7  assertTrue(user.isActive());
8  assertNotNull(user.getCreatedAt());
9  // اینجا ۴ تا چیز مختلف رو تست کرده
10 }
11
12 // b. خوب - هر تست فقط به کار 
13 @Test
14 void shouldCreateUserWithCorrectUsername() {
15     User user = service.register("ali", "pass");
16     assertEquals("ali", user.getUsername());
17 }
18
19 @Test
20 void shouldSetUserActiveByDefault() {
21     User user = service.register("ali", "pass");
22     assertTrue(user.isActive());
23 }
24
25 @Test
26 void shouldSetCreatedDateOnRegistration() {
27     User user = service.register("ali", "pass");
28     assertNotNull(user.getCreatedAt());
29 }
```

۵. اصل نام‌گذاری گویا

۵. اصل نام‌گذاری گویا اسم تست باید مشخص کنه چه چیزی رو تست میکنه، تحت چه شرایطی، و نتیجه‌ی مورد انتظار چیه.

```

1 // a. ✓ نام‌گذاری خوب
2 @Test
3 void shouldReturnFalseWhenPasswordIsTooShort() {}
4
5 @Test
6 void shouldThrowExceptionWhenUserNotFound() {}
7
8 @Test
9 void shouldCalculateDiscountCorrectlyForSeniorCitizens() {}
10
11 @Test
12 void shouldNotAllowDuplicateUsernames() {}
13
14 // b. ✗ نام‌گذاری بد
15 @Test
16 void test1() {}
17
18 @Test
19 void testLogin() {} // معلوم نیست چی رو تست میکنه
20
21 @Test
22 void test() {} // اصلاً چی؟

```


۶. اصل جداسازی دغدغه‌ها (Separation of Concerns)

قانون: تست‌های واحد باید فقط یک تابع یا کلاس رو تست کنن و بقیه وابستگی‌ها رو با Mock جایگزین کنی.

۱۶.۶ تست در جاوا با JUnit 5

JUnit 5 محبوب‌ترین فریم‌ورک تست در جاواست.

۱۶.۶.۱ مفاهیم پایه JUnit

```
1 import org.junit.jupiter.api.*;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class CalculatorTest {
5
6     private Calculator calculator;
7
8     @BeforeAll
9     static void beforeAll() {
10         System.out.println("قبل از همه تست‌ها");
11     }
12
13     @AfterAll
14     static void afterAll() {
15         System.out.println("بعد از همه تست‌ها");
16     }
```

```

17
18 @BeforeEach
19 void setUp() {
20     calculator = new Calculator();
21 }
22
23 @AfterEach
24 void tearDown() {
25     System.out.println("بعد از هر تست");
26 }
27
28 @Test
29 @DisplayName("باید جمع دو عدد رو درست محاسبه کنه")
30 void shouldAddTwoNumbersCorrectly() {
31     int a = 5;
32     int b = 3;
33     int result = calculator.add(a, b);
34     assertEquals(8, result, "جمع ۵ و ۳ باید ۸ بشه");
35 }
36 @Test
37 void shouldThrowExceptionWhenDividingByZero() {
38     assertThrows(ArithmeticException.class,
39         () -> calculator.divide(10, 0));
40 }
41 }

```

۱۶.۶.۲ Assertion های مهم JUnit

1 // بررسی برابری a.

```
2 assertEquals(4, calculator.add(2, 2));
3 assertEquals(3.14, PI, 0.001);
4
5 // b. بررسی نابرابری
6 assertNotEquals(5, calculator.add(2, 2));
7
8 // c. بودن null بررسی
9 assertNull(user);
10 assertNotNull(user);
11
12 // d. بررسی شرط
13 assertTrue validator.isValid("ali");
14 assertFalse validator.isValid("");
15
16 // e. بررسی استثنا
17 assertThrows(ArithmeticException.class,
18     () -> calculator.divide(10, 0));
19
20 // f. بررسی همه شرطها
21 assertAll("user validation",
22     () -> assertNotNull(user),
23     () -> assertTrue(user.isActive()),
24     () -> assertEquals("ali", user.getUsername())
25 );
26
27 // g. بررسی آرایهها
```

```

28 assertEquals(expectedArray, actualArray);
29
30 // h. بررسی timeout
31 setTimeout(Duration.ofSeconds(1),
32     () -> service.heavyOperation());

```

۱۶.۶.۳ تست‌های پارامتری (Parameterized Tests)

```

1 import org.junit.jupiter.params.ParameterizedTest;
2 import org.junit.jupiter.params.provider.*;
3
4 class ParameterizedTestExample {
5
6     @ParameterizedTest
7     @ValueSource(ints = {2, 4, 6, 8, 10})
8     void shouldReturnTrueForEvenNumbers(int number) {
9         assertTrue(calculator.isEven(number));
10    }
11
12    @ParameterizedTest
13    @CsvSource({
14        "5, 3, 8",
15        "10, 20, 30",
16        "-5, 3, -2"
17    })
18    void shouldAddNumbersCorrectly(int a, int b, int expected) {
19        assertEquals(expected, calculator.add(a, b));

```

```

20 }
21
22 @ParameterizedTest
23 @EnumSource(DayOfWeek.class)
24 void shouldHaveCorrectDayName(DayOfWeek day) {
25     assertNotNull(day.name());
26 }
27 }

```

۱۶.۷ Mock و Stub (شبیه‌سازی)

چرا؟ وقتی به کلاس وابستگی به دیتابیس یا سرویس خارجی داره، برای تست باید اون وابستگی رو شبیه‌سازی کنیم.

۱۶.۷.۱ تفاوت Mock و Stub

مفهوم	توضیح	پیچیدگی	کاربرد
Stub	یه پیاده‌سازی ساده که همیشه به مقدار ثابت برمی‌گردونه	ساده	وقتی فقط به یه مقدار خاص نیاز داری
Mock	یه شیء شبیه‌سازی شده که می‌تونی روش Expectation تعریف کنی و Verify کنی	پیچیده	وقتی می‌خواهی رفتار رو کنترل و بررسی کنی

۱۶.۷.۲ مثال Stub (ساده)

```

1 class EmailServiceStub implements EmailService {
2     @Override
3     public boolean sendEmail(String to, String body) {
4         return true;

```

```

5  }
6  }
7
8  class UserServiceTest {
9      private UserService service;
10
11     @BeforeEach
12     void setUp() {
13         EmailService emailService = new EmailServiceStub();
14         service = new UserService(emailService);
15     }
16
17     @Test
18     void shouldRegisterUser() {
19         assertTrue(service.register("ali", "pass"));
20     }
21 }

```

۱۶.۷.۳ مثال Mock با Mockito

```

1  import org.mockito.*;
2  import static org.mockito.Mockito.*;
3
4  class UserServiceTest {

```

```
5
6  @Mock
7  private UserRepository repository;
8
9  @Mock
10 private EmailService emailService;
11
12 @InjectMocks
13 private UserService service;
14
15 @BeforeEach
16 void setUp() {
17     MockitoAnnotations.openMocks(this);
18 }
19
20 @Test
21 void shouldReturnUserWhenFound() {
22     User mockUser = new User("ali", "pass");
23     when(repository.findByUsername("ali"))
24         .thenReturn(Optional.of(mockUser));
25
26     User result = service.findUser("ali");
27
28     assertNotNull(result);
29     assertEquals("ali", result.getUsername());
30     verify(repository).findByUsername("ali");
```

```

31 }
32
33 @Test
34 void shouldThrowExceptionWhenUserNotFound() {
35     when(repository.findByUsername("unknown"))
36         .thenReturn(Optional.empty());
37
38     assertThrows(UserNotFoundException.class,
39         () -> service.findUser("unknown"));
40 }
41
42 @Test
43 void shouldSendEmailAfterRegistration() {
44     User user = new User("ali", "pass");
45     when(repository.save(any(User.class))).thenReturn(user);
46
47     service.register("ali", "pass");
48
49     verify(emailService).sendEmail(eq("ali"), anyString());
50     verify(emailService, times(1)).sendEmail(any(), any());
51 }
52 }

```

۱۶.۷.۴ متدهای پرکاربرد Mockito

```

1 // تنظیم رفتار a.
2 when(mock.method()).thenReturn(value);

```



```

3 when(mock.method()).thenThrow(exception);
4 when(mock.method()).thenReturn(value1).thenReturn(value2);
5 doThrow(exception).when(mock).voidMethod();
6
7 // b. Matcher
8 when(repository.findById(anyLong())).thenReturn(user);
9 when(repository.findById(eq(1L))).thenReturn(user);
10 when(repository.findByUsername(anyString())).thenReturn(user);
11
12 // c. Verification
13 verify(mock).method();
14 verify(mock, times(2)).method();
15 verify(mock, never()).method();
16 verify(mock, atLeast(1)).method();
17 verify(mock, atMost(3)).method();
18 verifyNoMoreInteractions(mock);
19
20 // d. Spying
21 @Spy
22 private UserService realService;
23 when(realService.expensiveMethod()).thenReturn("mock result");

```

۱۶.۸ تست در Spring Boot

Spring Boot ابزارهای قدرتمندی برای تست دارد.

۱۶.۸.۱ تست لایه Controller (با MockMvc)

```
1 import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
2 import org.springframework.test.web.servlet.MockMvc;
3 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
4 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
5
6 @WebMvcTest(UserController.class)
7 class UserControllerTest {
8
9     @Autowired
10     private MockMvc mockMvc;
11
12     @MockBean
13     private UserService userService;
14
15     @Test
16     void shouldReturnUserWhenExists() throws Exception {
17         User user = new User(1L, "ali");
18         when(userService.findUser(1L)).thenReturn(user);
19
20         mockMvc.perform(get("/api/users/1")
21             .contentType(MediaType.APPLICATION_JSON))
22             .andExpect(status().isOk())
23             .andExpect(jsonPath("$.username").value("ali"))
24             .andExpect(jsonPath("$.id").value(1));
25     }
```

```

26
27 @Test
28 void shouldReturn404WhenUserNotFound() throws Exception {
29     when(userService.findUser(999L))
30         .thenReturn(new UserNotFoundException());
31
32     mockMvc.perform(get("/api/users/999"))
33         .andExpect(status().isNotFound());
34 }
35 }

```

۱۶.۸.۲ تست لایه Service (با Mock)

```

1 import org.springframework.boot.test.context.SpringBootTest;
2 import org.springframework.boot.test.mock.mockito.MockBean;
3
4 @SpringBootTest
5 class UserServiceTest {
6
7     @MockBean
8     private UserRepository repository;
9
10    @Autowired
11    private UserService service;
12
13    @Test
14    void shouldRegisterNewUser() {

```

```

15     User newUser = new User("ali", "pass123");
16     when(repository.save(any(User.class)))
17         .thenReturn(newUser);
18
19     User result = service.register("ali", "pass123");
20
21     assertNotNull(result);
22     verify(repository).save(any(User.class));
23 }
24 }

```

۱۶.۸.۳ تست لایه Repository (با دیتابیس واقعی)

```

1 import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
2
3 @DataJpaTest
4 class UserRepositoryTest {
5
6     @Autowired
7     private UserRepository repository;
8
9     @Test
10    void shouldFindUserByUsername() {
11        User user = new User("ali", "pass");
12        repository.save(user);
13
14        Optional<User> found = repository.findByUsername("ali");

```

```

15
16     assertTrue(found.isPresent());
17     assertEquals("ali", found.get().getUsername());
18 }
19 }

```

۱۶.۸.۴ تست Integration (کل سیستم)

```

1 import org.springframework.boot.test.context.SpringBootTest;
2 import org.springframework.boot.test.web.client.TestRestTemplate;
3 import org.springframework.boot.test.web.server.LocalServerPort;
4
5 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
6 class IntegrationTest {
7
8     @LocalServerPort
9     private int port;
10
11     @Autowired
12     private TestRestTemplate restTemplate;
13
14     @Test
15     void shouldRegisterAndLoginUser() {
16         User user = new User("ali", "pass");
17         ResponseEntity<Void> registerResponse = restTemplate
18             .postForEntity("http://localhost:" + port + "/api/register",
19                 user, Void.class);

```

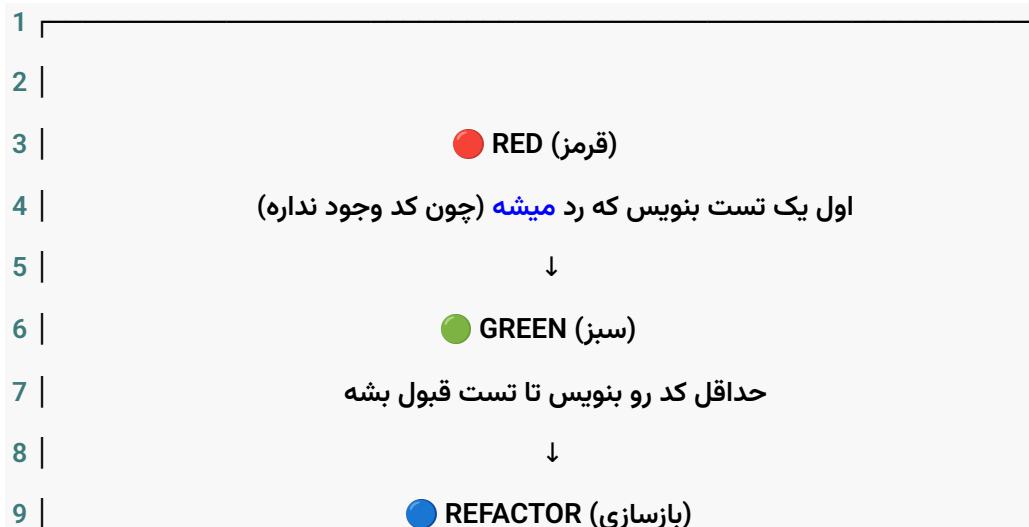
```

20 assertEquals(HttpStatus.CREATED, registerResponse.getStatusCode());
21
22 LoginRequest request = new LoginRequest("ali", "pass");
23 ResponseEntity<TokenResponse> loginResponse = restTemplate
24     .postForEntity("http://localhost:" + port + "/api/login",
25         request, TokenResponse.class);
26 assertEquals(HttpStatus.OK, loginResponse.getStatusCode());
27 assertNotNull(loginResponse.getBody().getToken());
28 }
29 }

```

۱۶.۹ TDD (Test-Driven Development)

چرخه (Red-Green-Refactor): TDD



16-Java-Testing

10		کد رو بهینه کن و تمیزش کن (تست‌ها باید سبز بمونن)	
11		↓	
12		(برگرد به مرحله ۱ برای ویژگی بعدی)	
13			
14			

مثال کامل TDD:

```
1 // a. ۱ مرحله: RED
2 @Test
3 void shouldReturnTrueForEvenNumber() {
4     assertTrue validator.isEven(4);
5 }
6
7 // b. ۲ مرحله: GREEN
8 public class NumberValidator {
9     public boolean isEven(int number) {
10         return number % 2 == 0;
11     }
12 }
13
14 // c. ۳ مرحله: REFACTOR
15 // (در صورت نیاز)
16
17 // ویژگی بعدی
18 @Test
19 void shouldReturnTrueForOddNumber() {
```

```

20 assertTrue validator.isOdd(5));
21 }

```

۱۶.۱۰ Coverage (پوشش تست)

سوال مصاحبه‌ای: Code Coverage چیست و چقدر باید باشد؟

جواب: درصد کدی که توسط تست‌ها پوشش داده شده. ابزارهایی مثل JaCoCo این رو اندازه می‌گیرن.

انواع Coverage:

• Line Coverage: چند درصد خطوط کد اجرا شدن

• Branch Coverage: چند درصد شرط‌ها (if/else) تست شدن

• Method Coverage: چند درصد متدها تست شدن

توصیه:

• هدف: ۷۰-۸۰٪ برای پروژه‌های معمولی

• ۱۰۰٪ همیشه ممکن نیست و گاهی هزینه‌بره

• کیفیت تست مهم‌تر از تعدادشه

```

1 <plugin>
2   <groupId>org.jacoco</groupId>
3   <artifactId>jacoco-maven-plugin</artifactId>
4   <configuration>
5     <rules>
6       <rule>
7         <element>CLASS</element>

```



```

8      <limits>
9      <limit>
10         <counter>LINE</counter>
11         <value>COVEREDRATIO</value>
12         <minimum>0.80</minimum>
13     </limit>
14 </limits>
15 </rule>
16 </rules>
17 </configuration>
18 </plugin>

```

۱۶.۱۱ خلاصه یک صفحه‌ای برای مصاحبه

خلاصه مفاهیم کلیدی تست نرم‌افزار

مفهوم	خلاصه
Test Pyramid	Unit > Integration > E2E (هرچه بالاتر، تعداد تست‌ها کمتر)
FIRST	Fast, Independent, Repeatable, Self-Validating, Timely
Given-When-Then	ساختار استاندارد نوشتن تست (زمینه - اقدام - انتظار)
Boundary Values	همیشه مرزها رو تست کن (۱۸، ۶۰، ۰، ۱۲۰، ...)
Negative Testing	ورودی‌های غلط و غیرمنتظره رو هم تست کن
Mock vs Stub	Mock قابل برنامه‌ریزی با انتظار رفتاری، Stub پاسخ‌دهنده ساده
TDD	Red ← Green ← Refactor (چرخه توسعه تست‌محور)
JUnit	فریم‌ورک اصلی تست در جاوا
Mockito	کتابخانه اصلی Mock در جاوا
SpringBootTest@	تست کامل Spring Boot با بارگذاری کل Context

مفهوم	خلاصه
DataJpaTest@	تست لایه دیتابیس (تنها Repository ها و Hibernate)
WebMvcTest@	تست لایه Controller (تنها Web Layer)
Coverage	هدف: ۷۰-۸۰٪ (کیفیت مهم تر از درصد)

۱۶.۱۲ سوالات رایج مصاحبه‌ای تست

? سوال ۱: فرق Unit Test و Integration Test چیه؟

جواب:

- Unit Test: یک واحد کوچک رو به تنهایی تست میکنه. وابستگی‌ها رو Mock میکنه. سریع و ارزونه.
- Integration Test: ارتباط چند جزء رو با هم تست میکنه. از دیتابیس یا سرویس واقعی استفاده میکنه. کندتر و گرون‌تره.

? سوال ۲: چطور از تست‌های کند جلوگیری کنیم؟

جواب:

- تست‌های Unit رو از Integration جدا کن
- از @Profile("test") برای دیتابیس تستی استفاده کن
- از H2 (دیتابیس In-Memory) برای تست استفاده کن
- Mock کردن وابستگی‌های سنگین (مثل سرویس‌های خارجی)

? سوال ۳: فرق @Mock و @MockBean چیست؟

جواب:

• @Mock: از Mockito میاد و فقط به Mock ساده می‌سازه (بدون Spring)

• @MockBean: از Spring Boot میاد و به Mock رو توی Spring Context قرار میده (برای تست‌های Spring)

? سوال ۴: چگونه کلاس رو تست‌پذیر کنیم؟

جواب:

• وابستگی‌ها رو از طریق Constructor تزریق کن (نه new کردن داخل کلاس)

• از Interface‌ها استفاده کن تا بتونی Mock کنی

• متدها رو کوتاه و با یک مسئولیت بنویس

• از Static method‌ها تا حد امکان کمتر استفاده کن